CY16 Binary Utilities Reference

Overview

The EZ-OTG/EZ-Host development kit includes a small set of binary utilities to perform simple functions during the development process. This document describes the use of some of these key utilities, which are WIN32 console mode programs. All input parameters are in either hex (with a leading 0x), decimal (no leading characters), or binary (with a leading 0b).

For the "QT" utilities, programs that start with "QTS" are designed to work over a serial port. Those that start with "QTU" work over USB. Where there are both serial and USB versions, they share the same basic syntax and function. Any differences will be noted below. The parameters for the QTS programs are stored in QTSERIAL.INI. The .INI file must remain in the same directory as the utilities.

There are some additional notes for use of the USB utilities. The "QTU" utilities will require the use of the cyusbgen.sys driver, included in the "Drivers" folder of the development kit. The QTU utilities also rely on the EZ-OTG and EZ-Host BIOS to perform their functions. As a result, the BIOS Idle_Task must be running. This is the default case, but custom user code must not override this in order to use these utilities (see "USB Multi-Role Device Design By Example" and the EZ-OTG / EZ-Host Technical Reference Manual for more details on BIOS functionality). Finally, there must be an available USB port that is not used by the application.

In the Usage sections below, items inside <..> are required and items inside [..] are optional.

SCANWRAP

Overview

EZ-OTG and EZ-Host have the ability to read a program from an I^2C EEPROM, from the UART, from the USB port, and from ROM and perform various operations with it. These operations include copying to RAM, initializing interrupt vectors, jumping or calling an absolute address, etc. In order for the EZ-OTG/EZ-Host BIOS to do this, the image must be prepared with special headers that contain SCAN signatures. These headers must be compatible with Interrupt 67 (SCAN_INT), which is described in the EZ-OTG / EZ-Host BIOS User's Manual. The headers can be embedded in your code, or they can be added to a binary image by using the scanwrap.exe utility.

Usage

SCANWRAP <in_file> <out_file> <base_address>

Where:

<in_file>	is the name of the input file
<out_file>	is the designated output file name
<base_address>	is the base address where the program is to be loaded

Notes:

The code beginning at <base_address> should perform initialization and return so that the BIOS may complete it's initialization and begin idle task processing.
The address used must be the same as that used in the linker script because SCANWRAP does not relocate code.

Example

SCANWRAP de1.bin de1_scan.bin 0x00001000

The following is a sample of the program output:

inpath = de1.bin
outpath = de1_scan.bin
base_address = 0x00001000
in_file_size = 17886
out_file_size = 17941

In this example de1_scan.bin has had the appropriate SCAN signatures added to it, and is ready to be loaded into an I^2C EEPROM, loaded over USB, etc.

Note
If you run scanwrap.exe without any command line arguments, it will output a help screen on your console.

QTSDUMP & QTUDUMP

Overview

The programs QTSDUMP & QTUDUMP are used to display the memory of the CY16 processor (i.e. EZ-OTG and EZ-Host). The input parameters are the location in memory and length to display. The output will be the contents of the memory dump in hexadecimal format.

Usage

QTSDUMP <address> [length]
QTUDUMP <address> [length]

Where:

<address>	is a value from 0x0000 to 0xFFFF.
[length]	is a value from 0x0000 to 0xFFFF. (default = 32 bytes)

Note:
Using QTSDUMP on the internal registers of the CY16 (i.e. 0xC000 to 0xC100) is not allowed. The contents displayed will not be correct. Use the QTUDUMP utility instead.

Example

```
QTUDUMP 0x4000 100      ; display 100 bytes external RAM memory
QTSDUMP 0xC000          ; display 32 bytes of internal CY16 registers at 0xC000
```

QTSARENA & QTUARENA

Overview

The programs QTSARENA & QTUARENA display the current memory use of the EZ-OTG/EZ-Host BIOS and any downloaded programs. At power-up, the EZ-OTG/EZ-Host BIOS examines all the available memory. The BIOS looks for available internal RAM and external RAM, if available. The QTSARENA & QTUARENA output shows the start of available memory and the size in bytes.

Usage

QTSARENA
QTUARENA

Example

QTSARENA

The following is a sample of the program output:

com1: baud=28800 parity=N data=8 stop=1 BIOS_Rev:7 Core_Rev: a
start 04a4 size 7b58 Empty
start 8000 size 8000 Used

The breakdown of each line is as follows:

com1: baud=28800 parity=N data=8 stop=1 BIOS_Rev:7 Core_Rev: a
com1: The current setup in this example is using COM port 1.
baud: This setup is shown in the QTSERIAL.INI
parity=N, data=8, stop=1 parameters are the required parameters.
BIOS Revision: 7 Current release version of the BIOS
Core_Rev: Hardware CY16 Core revision

start 04a4 size 7b58 Empty
The memory area starting at 0x04A4 (Internal RAM of EZ-OTG/EZ-Host) with the size 0x7B58 bytes is available. In this example, there is external RAM attached. The BIOS will determine if it can be used.

start 8000 size 8000 Used
The memory area starting at 0x8000 with the size 0x8000 bytes is not accessible by the EZ-OTG/EZ-Host.

QTUARENA

The following is a sample of the program output:

DEV: \\.\USBSCAN0 VID=4b4 PID=7200 FW_Ver:0009 BIOS_Rev:6 Core_Rev:9
start 04a4 size 7b58 Empty
start 8000 size 8000 Used

The breakdown of each line is as follows:

DEV: \\.\USBSCAN0 is the Cypress WDM Device Name
The VID=0x4B4 is the Cypress Vendor ID
PID=0x7200 is the Cypress Product ID
All other data is the same as the QTSARENA output

Note: The output of these commands will be different when you download a new program into the EZ-OTG/EZ-Host memory.

QTUI2C

Overview

The QTUI2C utility is used to program an I^2C EEPROM that connects externally to the EZ-OTG or EZ-Host chip. QTUI2C supports code that was assembled at location zero (org 0x0000) or assembled at another non-zero location. It also allows programming the EEPROM with data such as USB data

descriptors. These USB data descriptors must include the SCAN signatures compatible with Interrupt 67 (SCAN_INT) as described above.

To program a file that is assembled at location zero, the input filename without the extension is required. For example: “QTUI2C filename”. QTUI2C will create a file “makeenh.bin” based on the object input file. This “makeenh.bin” file will contain the fix-up (.bin) information and it will have a proper format compatible with the BIOS.

To program the file that is assembled at a non-zero location, the input filename with the extension (.bin) is required, for example: “QTUI2C filename.bin”. This code must include the SCAN signatures.

When using EZ-OTG/EZ-Host in EEPROM mode, at boot-up the BIOS will scan the EEPROM for a valid signature. If detected, it will load the code and/or data to the internal/external RAM. Any code/data that is not compatible with the required BIOS format will not be loaded at power up.

Usage

QTUI2C <filename> [f (other options)]

Notes:
The other options are [I | f1 | f11 | f2], which are reserved for other Cypress processors.

QTUI2C filename with no options is used for programs that will be stored in 256-2K byte EEPROMs.

QTUI2C filename f is used for programs that will be stored in 4K-64K byte EEPROMs.

When “filename” is used without an extension, QTUI2C will look for “filename.obj” and “filename.fix”. Using these for input it will add scan signatures to create a download format compatible with the BIOS. Normally this code is compiled originating at location zero.

When “filename” includes the extension “.bin” , QTUI2C assumes it is raw code/data ready for download. In this case the file must already have the appropriate scan signatures added, for example “0xC3B6” at the beginning of the file. When building code with the GNU tools, the utility scanwrap.exe can be used to add the scan signatures.

This program is only used when an I^2C EEPROM is connected to the EZ-OTG/EZ-Host.

After the program has completed, the code/data will be stored in the EEPROM. At any subsequent boot up the BIOS will automatically scan the EEPROM and load the contents into internal/external RAM. The EZ-OTG/EZ-Host can then begin executing the downloaded code.

Sometimes the EEPROM can become corrupted and can cause an unexpected error in the USB communication. If you wish to reprogram the EEPROM, then you can short Pins 5 and 6 of the EEPROM during reset of the EZ-OTG/EZ-Host. If the BIOS does not recognize the presence of the EEPROM it will skip the loading process. After boot up, you can re-program the EEPROM

In some cases custom code may choose to reduce the EZ-OTG / EZ-Host clock speed. A side effect of this is that EEPROM access is also slowed down. In this situation, attempting to reprogram a large EEPROM image may take a significant amount of time. This process can be improved by reprogramming the first byte of the EEPROM using the procedure below – a relatively quick process. Reprogramming this byte removes the scan signature that the BIOS uses to tell it to load code from the EEPROM. The result is that the next time the chip is powered up it will ignore any custom code in the EEPROM and load the default BIOS. The remainder of the EEPROM can then be reprogrammed in the normal fashion.

1) Create a file ‘zero.asm’ that contain a single line “zero dw 0”.
2) Assemble the ‘zero.asm’ file via the development tools.
3) Reprogram the EEPROM using the command “QTUI2C zero.bin”.
4) Reset the EZ-OTG/EZ-Host board.
5) Run QTSARENA or QTUARENA to verify that the EEPROM is empty (The memory at address 0x04a4 should be free). If the EEPROM is empty then proceed to step 6, otherwise repeat step 1 to step 5. You can also short pins 5 and 6 of the EEPROM (as mentioned above) if necessary.
6) Run QTUI2C.

Examples

QTUI2C filename	Use for small EEPROM (16kbit or less)
QTUI2C filename f	Use for large EEPROM (32kbit or greater)

Note:
If you know exactly where the code needs to be located and design your code appropriately you can use the .bin file produced by the development tools. Programming the EEPROM using the binary output file (.bin) usually saves more space in the EEPROM than programming the file without the extension.